

Project Proposal and Architecture Report

An Intelligent Video Segmentation and Semantic Clustering System

Optimizing Frame Extraction, Vision-Language Embeddings,
and Hardware-Aware Execution

Institution: Cochin University of Science and Technology

Date: July 7, 2026

Submitted for academic review and thesis guidance

Contents

Abstract	1
1 Introduction	2
1.1 Problem Statement	2
1.2 Project Objectives	3
1.3 Simple Meaning of Important Terms	4
2 System Architecture and Base Pipeline	5
2.1 System Architecture Flowchart	5
2.2 A. Optimized Scene Boundary Detection	5
2.3 B. Multithreaded Frame Extraction	6
2.4 C. Semantic Clustering Engine	6
2.5 D. Non-Blocking Assembly Interface	7
3 Proposed Contributions and Research Scope	8
3.1 A. Model Efficiency and Quantization	8
3.1.1 Comparing Heavy and Lightweight Models	8
3.1.2 Quantization	9

3.2	B. Algorithmic Clustering Improvements	9
3.2.1	Dynamic Cluster Selection	9
3.2.2	Streaming or Online Clustering	10
3.3	C. Pipeline and Systems-Level Acceleration	10
3.3.1	Batch Inference	10
3.3.2	Embedding Cache	10
3.4	D. Hardware-Aware Execution	11
4	Evaluation Plan	12
4.1	Performance Metrics	12
4.2	Clustering Quality Metrics	12
5	Expected Outcomes	13
6	Conclusion	14

Abstract

This project is about building a smart video analysis system. The system takes a long video, finds important frames, understands the visual meaning of those frames, and groups similar scenes together automatically. This reduces the manual effort needed to search through long videos.

The main technical idea is to make video processing faster. Instead of reading every frame one by one using OpenCV, the system uses FFprobe to quickly find useful I-frame timestamps from the video file. Then FFmpeg extracts those frames using fast seeking. After that, CLIP converts each frame into a numerical representation called an embedding. Similar embeddings are grouped using clustering methods such as K-Means.

The final output is shown in a PyQt6 graphical interface. The interface runs heavy tasks in background threads so that the application does not freeze while videos are being processed. The proposed research also studies model quantization, better clustering methods, batch processing, caching, and hardware-aware execution.

The system finds important parts of a long video, understands them using AI, groups similar scenes, and shows them in an easy-to-use interface.
--

Chapter 1

Introduction

Videos contain a large number of frames. A short video may contain thousands of frames, and a long video may contain millions. Manually checking these frames is slow and difficult. For example, if a user wants to find all similar scenes in a long lecture, movie, meeting, or sports video, they may need to spend a lot of time watching and cutting clips manually.

This project solves that problem by creating an intelligent video segmentation and semantic clustering system. The system first finds useful frames from the video. Then it uses an AI model to understand what each frame contains. Finally, it groups similar frames or scenes together so that the user can review them easily.

1.1 Problem Statement

Traditional video processing is often slow because it reads video frames one after another. This is called sequential frame decoding. It is simple, but it wastes time when the system only needs some important frames instead of every frame.

The main problems are:

- Reading every frame takes a lot of time for long videos.
- Extracting frames one by one does not use the computer efficiently.
- A fixed number of clusters may not work well for every video.
- Large AI models such as CLIP can be slow on systems without a GPU.

- A graphical interface may freeze if heavy video and AI tasks run on the main thread.

1.2 Project Objectives

The objectives of the project are:

- To find important video timestamps quickly using FFprobe.
- To extract keyframes faster using FFmpeg and multithreading.
- To convert extracted frames into AI-based visual embeddings using CLIP.
- To group similar scenes using clustering algorithms such as K-Means.
- To build a PyQt6 interface that stays responsive while background tasks run.
- To study ways to make the system faster using quantization, caching, batching, and hardware detection.

1.3 Simple Meaning of Important Terms

Term	Simple Meaning
FFprobe	A tool that reads video file information without fully opening every frame.
FFmpeg	A tool used to cut, convert, and extract frames from videos.
I-frame	A complete video frame that can be decoded independently.
CLIP	An AI model that understands images and text in a shared meaning space.
Embedding	A list of numbers that represents the meaning of an image or frame.
K-Means	A clustering method that groups similar data points together.
HDBSCAN	A clustering method that can find groups without fixing the number of clusters first.
PyQt6	A Python library used to build desktop graphical user interfaces.

Table 1.1: Simple explanation of major technical terms.

Chapter 2

System Architecture and Base Pipeline

The system works like a pipeline. A pipeline means that the output of one step becomes the input of the next step. In this project, the video first goes through timestamp detection, then frame extraction, then AI embedding, then clustering, and finally display in the user interface.

2.1 System Architecture Flowchart

Figure 2.1 shows the main flow of the system.

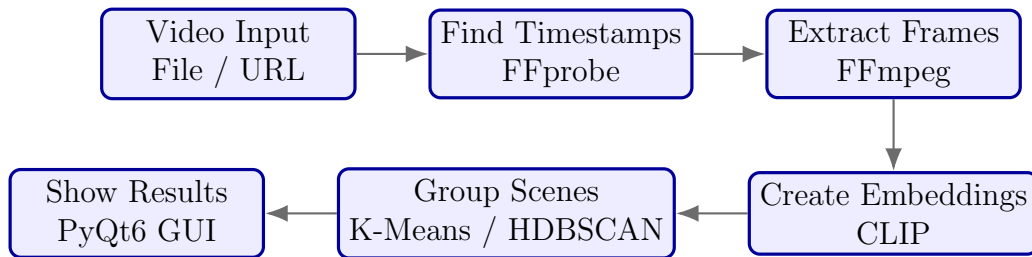


Figure 2.1: Simple system architecture of the proposed video analysis pipeline.

2.2 A. Optimized Scene Boundary Detection

The first step is to find useful positions in the video. A normal method is to read frames one by one using OpenCV. This is slow because the system has to decode many frames

even if only a few keyframes are needed.

The proposed method uses FFprobe. FFprobe can inspect the internal structure of a video file and find timestamps such as `pts_time`. These timestamps help identify useful I-frames. Since I-frames are complete frames, they are good candidates for extracting important visual information.

Instead of watching the whole video frame by frame, the system reads the video's internal timestamp information and quickly finds useful frame positions.

```
ffprobe -select_streams v -show_packets -show_entries packet=pts_time,flags
input.mp4
```

Listing 2.1: Example FFprobe command for reading packet timestamps.

2.3 B. Multithreaded Frame Extraction

After finding useful timestamps, the next step is to extract frames from those positions. The system uses FFmpeg with the `-ss` option. This option allows FFmpeg to jump directly to a timestamp and extract a frame.

To make this faster, multiple frame extraction tasks are run at the same time using a `ThreadPoolExecutor`. This means the system does not wait for one frame to finish before starting the next one.

The system finds many useful timestamps and extracts frames from them in parallel, which saves time.

```
ffmpeg -ss <timestamp> -i input.mp4 -frames:v 1 -q:v 2 keyframe.jpg
```

Listing 2.2: Example FFmpeg command for fast frame extraction.

2.4 C. Semantic Clustering Engine

After the frames are extracted, the system needs to understand what each frame represents. For this, it uses CLIP. CLIP converts each frame into an embedding. An embedding is a numerical form of the image's meaning.

Once embeddings are created, clustering is used to group similar scenes together. For example, all frames showing a classroom may form one group, while all frames showing outdoor scenes may form another group. The basic method is K-Means, and improved methods such as HDBSCAN can also be studied.

CLIP turns each image into numbers that represent meaning. Clustering then puts similar scenes into the same group.

2.5 D. Non-Blocking Assembly Interface

The system uses a PyQt6 graphical interface so users can view and organize the grouped scenes. Video processing and AI processing can take time, so these tasks should not run directly on the main interface thread.

To keep the interface smooth, the system uses background workers such as `QThread` and `QRunnable`. These workers run heavy tasks separately and send progress updates to the interface. As a result, the user can still interact with the application while processing is happening.

Heavy tasks run in the background, so the application does not freeze while processing videos.

Chapter 3

Proposed Contributions and Research Scope

This chapter explains the main research value of the project in simple terms. The project is not only a video tool; it also studies how to make video analysis faster, smarter, and more suitable for different hardware systems.

3.1 A. Model Efficiency and Quantization

AI models such as CLIP are useful, but they can be heavy. A heavy model may give better results, but it may run slowly on a normal laptop. A lighter model may run faster, but its accuracy may be lower. This project studies that speed-versus-quality tradeoff.

3.1.1 Comparing Heavy and Lightweight Models

The project can compare a standard model such as `clip-ViT-B-32` with lighter models such as MobileCLIP or TinyCLIP.

The comparison will check:

- How fast each model creates embeddings.
- How much memory each model uses.
- How good the final scene groups are.

- Which model is best for normal CPU-only systems.

3.1.2 Quantization

Quantization means reducing the numerical precision of a model. For example, instead of using large floating-point numbers, the model can use smaller INT8 numbers. This can make the model faster and smaller.

The project can test quantization using `torch.quantization` or ONNX Runtime. The goal is to make inference faster, ideally by 2–4x, while keeping the clustering result useful.

Quantization makes the AI model lighter so that it can run faster on ordinary computers.

3.2 B. Algorithmic Clustering Improvements

The first version of the system can use K-Means. K-Means is simple and fast, but it needs the number of clusters to be given in advance. For example, if $K = 5$, the system will always create five groups, even if the video naturally contains three or ten main events.

To improve this, the project can study smarter clustering methods.

3.2.1 Dynamic Cluster Selection

Dynamic cluster selection means allowing the system to choose the number of groups automatically. This can be done using methods such as the Silhouette score or HDBSCAN.

- The Silhouette score helps measure how good a clustering result is.
- The Elbow method can help choose a useful value of K .
- HDBSCAN can find clusters without manually setting K .

3.2.2 Streaming or Online Clustering

In a normal pipeline, clustering starts only after all frames are extracted and all embeddings are created. Streaming clustering improves this by starting earlier. As soon as some embeddings are ready, the system can begin forming clusters.

Instead of waiting until the whole video is processed, the system can start grouping scenes as soon as enough frames are available.

3.3 C. Pipeline and Systems-Level Acceleration

This part focuses on making the whole system faster, not just the AI model.

3.3.1 Batch Inference

Batch inference means processing many frames together instead of one frame at a time. This usually improves performance because the model and hardware are used more efficiently.

Batch inference is like carrying many items in one trip instead of making one trip for each item.

3.3.2 Embedding Cache

Creating embeddings is expensive. If the same frame is processed again, the system should not repeat the same work. An embedding cache stores previously computed embeddings using a frame hash. If the user changes clustering settings, the system can reuse cached embeddings instead of running CLIP again.

- A frame hash works like a unique ID for a frame.
- Cached embeddings save time during repeated experiments.
- Users can change clustering settings without recomputing everything.

Caching remembers old results so the system does not waste time doing the same AI calculation again.

3.4 D. Hardware-Aware Execution

Different users may have different hardware. Some may have an NVIDIA GPU, some may have Apple Silicon, and some may only have a CPU. The system should detect the available hardware and choose the best execution method automatically.

The system can check for:

- **CUDA:** Used for NVIDIA GPU acceleration.
- **MPS:** Used for Apple Silicon acceleration.
- **CPU:** Used when no GPU acceleration is available.
- **ONNX Runtime:** Used for optimized model execution across platforms.
- **TensorRT:** Used for faster NVIDIA-specific inference.

The system should automatically choose the fastest available hardware option, but still work even on CPU-only machines.

Chapter 4

Evaluation Plan

The system must be tested to prove that it is useful and faster than basic methods. The evaluation will compare speed, memory use, and clustering quality.

4.1 Performance Metrics

- **Frame extraction time:** How long it takes to find timestamps and extract keyframes.
- **Embedding time:** How long CLIP takes to convert frames into embeddings.
- **Clustering time:** How long K-Means, HDBSCAN, or auto-K methods take.
- **Memory usage:** How much RAM or GPU memory the system uses.
- **Total processing time:** How long the full process takes from video input to grouped scenes.

4.2 Clustering Quality Metrics

- Silhouette score to check how well the clusters are separated.
- K-Means inertia to check how compact the clusters are.
- Manual review to check whether the grouped scenes make sense to humans.
- Comparison between fixed K-Means and dynamic clustering methods.

Chapter 5

Expected Outcomes

The expected result is a working research prototype that can process long videos, extract useful frames, understand their visual meaning, group similar scenes, and show the results in a responsive interface.

The project is expected to produce:

- A faster FFprobe and FFmpeg-based keyframe extraction pipeline.
- A CLIP-based embedding module for understanding visual meaning.
- A clustering module using K-Means and possible improved methods such as HDB-SCAN.
- A PyQt6 interface that stays responsive during background processing.
- Benchmark results for different models, hardware options, and optimization methods.

Chapter 6

Conclusion

This project proposes a simple but powerful idea: process videos faster, understand frames using AI, and group similar scenes automatically. The system avoids slow sequential frame decoding by using FFprobe and FFmpeg. It uses CLIP to convert frames into meaningful embeddings and clustering algorithms to group similar scenes.

The research value comes from studying how to make the pipeline faster and more practical. This includes model quantization, dynamic clustering, batch inference, embedding caching, and hardware-aware execution. The final system can support video summarization, highlight extraction, scene organization, and future research in intelligent video analysis.

The final goal is to build a system that can take a long video and automatically organize its important scenes in a fast and user-friendly way.
